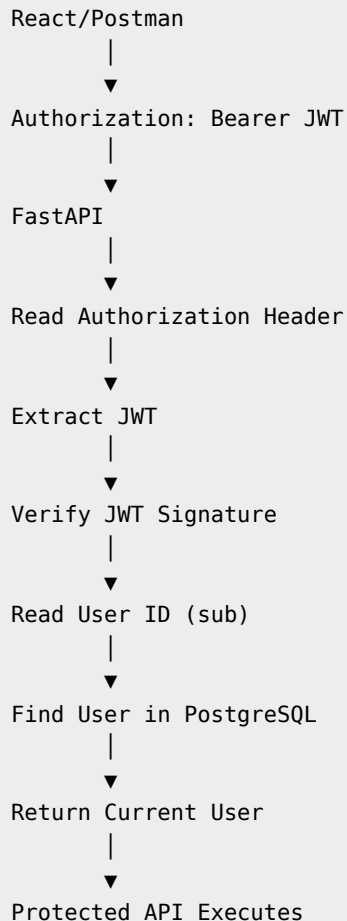


# Authorization Header

By the end of this lesson, you'll have this flow:



## Step 1: How the Authorization Header Works

When the client logs in:

POST /login

↓

JWT Returned

↓

eyJhbGc...

Now every request includes:

GET /profile

Authorization: Bearer eyJhbGc0iJIUzI1Ni...

The backend receives this header.

## Step 2: FastAPI's OAuth2PasswordBearer

FastAPI provides a helper that extracts the JWT from the `Authorization` header.

Create a new file:

dependencies.py

Import:

```
from fastapi.security import OAuth2PasswordBearer
```

Create:

```
oauth2_scheme=OAuth2PasswordBearer(tokenUrl="login")
```

---

## What does this do?

When a request arrives:

```
Authorization: Bearer abc.xyz.123
```

FastAPI automatically extracts:

```
abc.xyz.123
```

You don't have to manually parse the header.

Think of it like:

Request

↓

Authorization Header

↓

OAuth2PasswordBearer

↓

Token

Step 3: Create `get_current_user()`

Create the database dependency if you haven't already:

```
def get_db():
    db=SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Now create:

Notice:

```
token=Depends(oauth2_scheme)
```

FastAPI automatically gives you the JWT from the header.

```
def get_current_user(
    token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)
):
    # Logic to decode the token and retrieve the current user
    # from the database
    # Then, query the database using the provided db session
    # to get the user details
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

        user_id: int = payload.get("sub")

        if user_id is None:
            raise HTTPException(status_code=401, detail="Invalid Token")

    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")

    user = get_user_by_id(db, user_id)

    if user is None:
        raise HTTPException(status_code=401, detail="User not found")

    return user
```

## Step 4: Decode the JWT

Inside `get_current_user()`:

```
try:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

    user_id: int = payload.get("sub")

    if user_id is None:
        raise HTTPException(status_code=401, detail="Invalid Token")

except JWTError:
    raise HTTPException(status_code=401, detail="Invalid token")
```

Suppose the token contains:

```
{
  "sub": "1",
  "email": "sayyami@gmail.com"
}
```

After decoding:

```
payload
```

becomes:

```
{"sub": "1", "email": "sayyami@gmail.com", "exp": ...
}
```

## Step 5: Read the User ID

```
user_id=payload.get("sub")
```

Remember:

When logging in, we created:

```
{"sub":str(db_user.id)}
}
```

Now:

```
user_id
```

contains:

```
1
```

---

## Step 6: Find User in PostgreSQL

Add a new function to `crud.py`:

```
def get_user_by_id(db,user_id):
    return (db.query(User).filter(User.id==user_id).first())
```

Equivalent SQL:

```
SELECT*FROM usersWHERE id=1;
```

Back in `get_current_user()`:

```
user=get_user_by_id(db,user_id
)
```

If not found:

```
if user is None:
    raise HTTPException(status_code=401, detail="User not found")
```

Otherwise:

```
return user
```

## Step 7: Protect an API

Create:

```
from fastapi import APIRouter
from fastapi import Depends
from dependencies import get_current_user

router = APIRouter()
```

Create:

```
@router.get("/profile")
def profile(
    current_user=Depends(get_current_user)):
    return current_user
```

That's it!

No JWT verification code inside the endpoint.

## What is `Depends()` Really Doing?

Think of it like a security guard.

Without `Depends()`:

User

↓

Profile API

↓

API must verify JWT

↓

API must query database

↓

API returns data

Every endpoint repeats the same logic.

With `Depends()` :

User

↓

Security Guard (Depends)

↓

JWT Verified

↓

User Loaded

↓

Profile API

↓

Return Data

The endpoint focuses only on business logic.

## Interview Questions



## 1. What is the Authorization header?

The `Authorization` header is an HTTP request header used to send authentication credentials from the client to the server. With JWT authentication, it usually contains a bearer token in the format:

```
Authorization: Bearer <JWT>
```

## 2. Why do we use `Bearer` ?

`Bearer` indicates that the client is presenting a bearer token. Whoever possesses a valid token can use it to access protected resources, so it must be transmitted and stored securely.

## 3. Why do we decode the JWT?

To verify that the token is authentic, has not been modified, has not expired, and to extract user information (such as the `sub` claim).

## 4. Why do we query PostgreSQL after decoding the JWT?

The JWT typically contains only user identifiers and claims, not the complete user record. Querying PostgreSQL ensures:

- The user still exists.
- You can access current user details.
- You can enforce business rules (for example, if an account has been disabled).

## 5. What does `Depends(get_current_user)` do?

`Depends(get_current_user)` tells FastAPI:

1. Extract the JWT from the request.
2. Decode and verify it.
3. Load the corresponding user from the database.

4. Inject that user into the endpoint.

This avoids duplicating authentication code in every protected route.